



AUDIT REPORT

February 2026

For



Table of Content

Executive Summary	03
Number of Security Issues per Severity	05
Summary of Issues	06
Checked Vulnerabilities	07
Techniques and Methods	09
Types of Severity	11
Types of Issues	12
Severity Matrix	13
 Medium Severity Issues	14
1. Emergency Withdrawal Blocked During Pause	14
 Low Severity Issues	15
2. Unbounded Administrative Control Over Minimum Withdrawal Amount	15
 Informational Issues	16
3. Floating Pragma	16
Functional Tests	17
Automated Tests	17
Threat Model	18
Closing Summary & Disclaimer	26



Executive Summary

Project Name	Puffpaw
Protocol Type	Custodial Vault
Project URL	https://www.puffpaw.xyz/
Overview	The Token contract is an upgradeable ERC20 token using UUPS and AccessControl, introducing an ADMIN_ROLE for privileged operations such as minting, pausing, and upgrades. It enforces a fixed MAX_SUPPLY, ensuring supply cannot exceed 100 billion tokens The Vault contract acts as a secure token custody and withdrawal system, supporting upgradeability, pausings. It uses ECDSA-based signature verification for authorized withdrawals, ensuring only valid off-chain approvals can trigger fund releases. AccessControl regulates administrative actions, while SafeERC20 ensures safe token transfers.
Audit Scope	The scope of this Audit was to analyze the PuffPaw Smart Contracts for quality, security, and correctness.
Source Code link	https://drive.google.com/drive/folders/1RFHEmE67aguYUazJZdcKc_JdW7l9inRm
Contracts in Scope	Token.sol Vault.sol
Branch	Main
Language	Solidity
Blockchain	Ethereum
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	5th January 2026 - 13th January 2026
Updated Code Received	4th February 2026
Review 2	4th February 2026



Fixed In

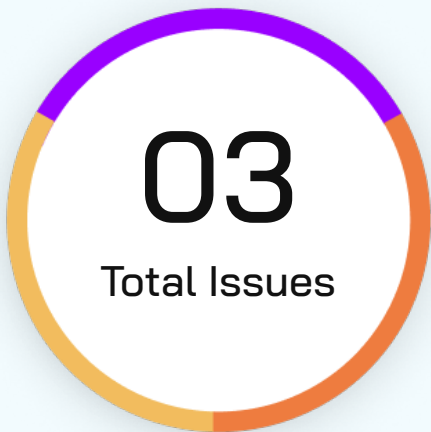
<https://github.com/puffpaw-lab/contracts/commit/05442212ccbb8c4626463f96cdce9f0b36559388>

Verify the Authenticity of Report on QuillAudits Leaderboard:

<https://www.quillaudits.com/leaderboard>



Number of Issues per Severity



Critical	0(0.0%)
High	0(0.0%)
Medium	1 (33.3%)
Low	1 (33.3%)
Informational	1 (33.3%)

		Severity				
		Critical	High	Medium	Low	Informational
Issues	Open	0	0	0	0	0
	Acknowledged	0	0	0	0	0
	Partially Resolved	0	0	0	0	0
	Resolved	0	0	1	1	1



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Emergency Withdrawal Blocked During Pause	Medium	Resolved
2	Unbounded Administrative Control Over Minimum Withdrawal Amoun	Low	Resolved
3	Floating Pragma	Informational	Resolved



Checked Vulnerabilities

✓ Access Management

✓ Arbitrary write to storage

✓ Centralization of control

✓ Ether theft

✓ Improper or missing events

✓ Logical issues and flaws

✓ Arithmetic Computations
Correctness

✓ Race conditions/front running

✓ SWC Registry

✓ Re-entrancy

✓ Timestamp Dependence

✓ Gas Limit and Loops

✓ Exception Disorder

✓ Gasless Send

✓ Use of tx.origin

✓ Malicious libraries

✓ Compiler version not fixed

✓ Address hardcoded

✓ Divide before multiply

✓ Integer overflow/underflow

✓ ERC's conformance

✓ Dangerous strict equalities

✓ Tautology or contradiction

✓ Return values of low-level calls



✓ **Missing Zero Address Validation**

✓ **Private modifier**

✓ **Revert/require functions**

✓ **Multiple Sends**

✓ **Using suicide**

✓ **Using delegatecall**

✓ **Upgradeable safety**

✓ **Using throw**

✓ **Using inline assembly**

✓ **Style guide violation**

✓ **Unsafe type inference**

✓ **Implicit visibility level**

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.



Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.



Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ **Critical: Immediate and Catastrophic Impact**

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ **High (H): Significant Risk of Major Loss or Compromise**

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ **Medium (M): Potential for Moderate Harm Under Specific Circumstances**

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

■ **Low (L): Minor Imperfections with Limited Repercussions**

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ **Informational (I): Opportunities for Improvement, Not Immediate Risks**

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.



Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

		Impact		
		High	Medium	Low
Likelihood	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.



Medium Severity Issues

Emergency Withdrawal Blocked During Pause

Resolved

Path

.contracts/Vault.sol

Function Name

`adminWithdraw`

Description

The `adminWithdraw` function is protected by the `whenNotPaused` modifier. While this modifier is generally appropriate for user-facing actions, it restricts the withdraw operator from executing emergency fund recovery when the contract is paused. In typical pause semantics, pausing is intended to stop normal operations while still allowing privileged roles to mitigate damage or rescue funds.

Impact - High

If the contract is paused due to an incident (e.g., exploit, misconfiguration, or external dependency failure), the withdraw operator will be unable to recover or relocate funds. This can result in funds being stuck in the contract during emergencies, increasing potential losses.

Likelihood

Medium. Pauses are commonly triggered during critical situations

POC

1. Admin pauses the contract.
2. Withdraw operator attempts to call `adminWithdraw`.
3. The call reverts due to `whenNotPaused`, preventing fund rescue.

Recommendation

Remove the `whenNotPaused` modifier from `adminWithdraw`. Alternatively, introduce a dedicated `emergencyWithdraw` function callable only by a trusted role during paused states.



Low Severity Issues

Unbounded Administrative Control Over Minimum Withdrawal Amount

Resolved

Path

.contracts/Vault.sol

Function Name

```
`setMinWithdrawAmount(uint256 newMinWithdrawAmount)`
```

Description

The `setMinWithdrawAmount` function allows an account with `ADMIN\_ROLE` to arbitrarily set the `minWithdrawAmount` to any value, including excessively high amounts. While administrative control is sometimes required, there are no constraints, caps, delays, or governance mechanisms limiting this power. As a result, the admin can effectively disable withdrawals or manipulate user behavior at will.

Impact - Medium

A malicious, compromised, or negligent admin can set `minWithdrawAmount` to a value higher than any user's balance, effectively blocking all withdrawals. This creates a strong centralization risk and undermines user trust, as access to funds ultimately depends on a single privileged role.

Likelihood

Low

Recommendation

Introduce safeguards such as:

- * A reasonable upper bound on `minWithdrawAmount`.
- * A timelock or delayed execution for changes.
- * Governance or multisig approval for parameter updates.



Informational Issues

Floating Pragma

Resolved

Path

.contracts/Vault.sol
.contracts/Token.sol

Description

Floating pragmas allow the contract to be compiled with multiple compiler versions. This can lead to unexpected behavior, as different compiler versions may introduce breaking changes, bug fixes, or different optimizations.

Impact - Low

Using a floating pragma can result in:

- Inconsistent bytecode across deployments
- Potential incompatibility with future compiler versions
- Increased risk during audits and production deployments

Likelihood

Low

Recommendation

Use a fixed Solidity compiler version instead of a floating pragma.



Functional Tests

Some of the tests performed are mentioned below:

Vault.sol

- ✓ Deposits correctly transfer tokens from users to vault and emit Deposited events with accurate amounts.
- ✓ Withdrawals with valid validator signatures properly verify nonce sequencing, requestId uniqueness, and deadline constraints before transferring tokens.
- ✓ Request IDs and user nonces prevent replay attacks by marking used requestIds and enforcing strictly incrementing nonces per user.
- ✓ AdminWithdraw function allows withdrawOperator to extract funds while enforcing recipient validation and balance checks.
- ✓ Pause functionality blocks all deposit and withdrawal operations while allowing admins to update critical parameters like validator and withdrawOperator addresses.

Token.sol

- ✓ Minting correctly increases supply while enforcing MAX_SUPPLY cap of 100 billion tokens and restricting access to ADMIN_ROLE.
- ✓ Withdraw function allows withdrawOperator to transfer tokens held by the contract to valid recipients while preventing transfers to zero or self address.
- ✓ WithdrawOperator changes can only be performed by ADMIN_ROLE and emit WithdrawOperatorChanged events with previous and new operator addresses.
- ✓ Contract upgrade authorization is restricted to ADMIN_ROLE through _authorizeUpgrade override ensuring only admins control implementation changes.
- ✓ Role-based access control properly segregates administrative functions from operator functions with appropriate revert messages for unauthorized access.

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.



Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
ERC20Upgradeable.sol	OpenZeppelin Library	Core token logic (balances, transfers, allowances)	Correct implementation of ERC20 invariants	Supply manipulation if flawed
IERC20 / SafeERC20	OpenZeppelin Library	Safe token transfers in Vault	External token contract is not malicious	Reentrancy, fake tokens, transfer hooks
AccessControlUpgradeable.sol	OpenZeppelin Library	Role-based access control (ADMIN_ROLE management)	Compiler has no known critical bugs	Older version can lead to unsafe defaults
Fee Recipient	External Address	Upgradeable proxy pattern implementation	Upgrade logic cannot be bypassed	Malicious upgrades, storage collision
ReentrancyGuardUpgradeable.sol	OpenZeppelin Library	Prevents reentrancy attacks in Vault	Guard is correctly implemented	Reentrancy if bypassed
PausableUpgradeable.sol	OpenZeppelin Library	Emergency pause mechanism in Vault	Pause state is respected	Griefing via pause
ECDSA.sol	OpenZeppelin Library	Signature verification for withdrawals	Signature cryptography is sound	Signature malleability, replay attacks



Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Initializable.sol	OpenZeppelin Library	Prevents re-initialization	Can only be initialized once	Double initialization attack
Solidity Compiler	Tool	Compilation (^0.8.19)	Compiler has no critical bugs for this version	Potential compiler-level vulnerabilities
Proxy Contract	External Contract	Delegates calls to implementation	Proxy correctly delegates and maintains storage	Storage layout corruption, selector clashing
External Token (in Vault)	Smart Contract	The token being deposited/withdrawn	Token is ERC20 compliant and not malicious	Deflationary tokens, fee-on-transfer, reentrancy hooks
Off-chain Validator	External Service	Signs withdrawal requests for Vault	Validator keys are secure, service is available	Service downtime, key compromise, censorship

2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets one table.

Contract Name	Function	Category
Token.sol	initialize()	What this function can do Initialize the contract once with token name, symbol, admin address, and withdrawOperator. Grants DEFAULT_ADMIN_ROLE and ADMIN_ROLE to admin. Sets withdrawOperator. What this function cannot/should not do Cannot be called twice due to the initializer modifier. Cannot set zero addresses for admin or withdrawOperator. Cannot mint tokens during initialization. Main invariant(s) Can only be called once. Admin and withdrawOperator must be non-zero addresses. Access level Public (but protected by initializer modifier - can only be called once during deployment)
	setWithdrawOperator()	What this function can do Change the withdrawOperator address to a new non-zero address. Emit WithdrawOperatorChanged event. What this function cannot/should not do Cannot set withdrawOperator to address(0). Cannot be called by non-ADMIN_ROLE holders. Main invariant(s) withdrawOperator is always a valid non-zero address. Only ADMIN_ROLE can modify.



Contract Name	Function	Category
	mint()	Access level Restricted (ADMIN_ROLE only) What this function can do Mint new tokens to any address up to MAX_SUPPLY (100 billion tokens). Increase totalSupply. What this function cannot/should not do Cannot exceed MAX_SUPPLY cap. Cannot be called by non-ADMIN_ROLE. Cannot mint to address(0) (inherited from ERC20).Main Main invariant(s) $\text{totalSupply() + amount} \leq \text{MAX_SUPPLY}$ at all times. Only ADMIN_ROLE can
	withdraw()	Access level Restricted (ADMIN_ROLE only) What this function can do Transfer tokens held by the contract itself to a specified address. Emit Withdraw event. What this function cannot/should not do Cannot transfer to address(0) or address(this). Cannot be called by anyone except the withdrawOperator. Cannot transfer more than contract's balance (reverts via ERC20). Main invariant(s) Only withdrawOperator can call. Recipients must be valid non-zero, non-self addresses. Access level Restricted (withdrawOperator only)

Contract Name	Function	Category
Vault.sol	<code>_authorizeUpgrade()</code>	What this function can do Authorize contract upgrades in UUPS pattern. Gates who can upgrade the implementation. What this function cannot/should not do Cannot be called by non-ADMIN_ROLE holders. Cannot bypass upgrade authorization. <code>_authorizeUpgrade</code> function is not compliant with OpenZeppelin because the overridden function does not match the parent function's mutability Main invariant(s) Only ADMIN_ROLE can authorize upgrades. Access level Internal (called by UUPS upgrade mechanism, restricted to ADMIN_ROLE)
	<code>deposit()</code>	What this function can do Pull tokens from msg.sender into the Vault. Emit Deposited event. What this function cannot/should not do Cannot deposit 0 amount. Cannot be called when paused. Cannot deposit if user hasn't approved tokens. Main invariant(s) Vault token balance increases by deposit amount. User has sufficient balance and approval. Access level Public (with nonReentrant and whenNotPaused guards)



Contract Name	Function	Category
	withdrawWithValidation()	What this function can do Pull tokens from msg.sender into the Vault. Emit Deposited event What this function cannot/should not do Cannot deposit 0 amount. Cannot be called when paused. Cannot deposit if user hasn't approved tokens. Main invariant(s) Vault token balance increases by deposit amount. User has sufficient balance and approval. Access level (with nonReentrant and whenNotPaused guards)
	adminWithdraw()	What this function can do Withdraw any amount from Vault to any address (except 0 or self). Emergency/administrative fund recovery. What this function cannot/should not do Cannot withdraw to address(0) or address(this). Cannot exceed vault balance. Cannot be called when paused. Cannot be called by non-withdrawOperator. Main invariant(s) Only withdrawOperator can call. Recipient must be valid. Access level Restricted (withdrawOperator only, with nonReentrant and whenNotPaused)



Contract Name	Function	Category
	<code>_authorizeUpgrade()</code>	What this function can do Authorize contract upgrades in UUPS pattern. Gates who can upgrade the implementation. What this function cannot/should not do Cannot be called by non-ADMIN_ROLE holders. Cannot bypass upgrade authorization. _authorizeUpgrade function is not compliant with OpenZeppelin because the overridden function does not match the parent function's mutability Main invariant(s) Only ADMIN_ROLE can authorize upgrades. Access level Internal (called by UUPS upgrade mechanism, restricted to ADMIN_ROLE)

3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets
Token.sol.	Native Token	Tokens held by contract itself (via transfers to <code>address(this)</code>) Unusual



3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
Token.sol	mint()	-	New tokens minted	ADMIN_ROLE	Inflation risk if admin compromised. Hard cap at MAX_SUPPLY provides some protection. No rate limiting.
Token.sol	withdraw()	-	Tokens held by contract	withdrawOperator	Inflation risk if admin compromised. Hard cap at MAX_SUPPLY provides some protection. No rate limiting.
Token.sol	transfer()	Tokens from sender	Tokens to recipient	Public	Standard ERC20 transfer risks. Could transfer to contract address accidentally.
Token.sol	transferFrom()	Tokens from approved account	Tokens to recipient	Public (with allowance)	Approval front-running, standard ERC20 risks



Closing Summary

In this report, we have considered the security of Puffpaw. We performed our audit according to the procedure described above.

No critical issues in Puffpaw, just 1 Medium, 1 Low and Information issues were found, Puffpaw resolved the issues mentioned above.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.



About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers.

With seven years of expertise, we've secured over 1400 projects globally, averting over \$3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

**7+**

Years of Expertise

1M+

Lines of Code Audited

50+

Chains Supported

1400+

Projects Secured

Follow Our Journey



AUDIT REPORT

February 2026

For



Canada, India, Singapore, UAE, UK

www.quillaudits.com

audits@quillaudits.com